

BURP SUITE

ACADEMY / RAPPORT D'INTERVENTION SQLi

Rapport de Laboratoire 1

Attaque SQL UNION et détermination du nombre de colonnes

Découverte itérative par `UNION SELECT NULL`

Étudiant	Gana Faye
Formation	Sécurité des applications web
Plateforme	PortSwigger Web Security Academy
Laboratoire	SQL injection UNION attack, determining the number of columns
Module	Injections SQL
Niveau	Practitioner
Fenêtre d'intervention	09:44 à 09:58
Date	28 juillet 2026
Statut	Résolu
Niveau de risque	Élevé

Table des matières

1	Introduction et contexte du laboratoire	2
1.1	Présentation de l'exercice	2
1.2	Objectif technique	2
1.3	Périmètre et résultat attendu	2
2	Phase de reconnaissance et interception	2
2.1	Étape 1 – Accès initial à la plateforme (09 :44)	3
2.2	Étape 2 – Analyse du trafic HTTP (09 :46)	3
2.3	Étape 3 – Transfert vers Burp Repeater (09 :47)	4
3	Phase d'exploitation et tâtonnement itératif	5
3.1	Principe de l'approche incrémentale	5
3.2	Étapes 4 et 5 – Test avec une colonne (09 :51–09 :52)	6
3.3	Étapes 6 et 7 – Test avec deux colonnes (09 :54–09 :55)	8
3.4	Étapes 8 et 9 – Validation avec trois colonnes (09 :57)	10
3.5	Synthèse comparative des essais	13
4	Clôture et validation du laboratoire	13
4.1	Étape 10 – Validation officielle (09 :58)	13
4.2	Chronologie de l'intervention	14
5	Analyse du risque	14
5.1	Vulnérabilité confirmée	14
5.2	Impacts potentiels	14
5.3	Sévérité	14
6	Recommandations techniques et remédiation	15
6.1	Utiliser des requêtes préparées	15
6.2	Valider les catégories autorisées	15
6.3	Appliquer le moindre privilège	15
6.4	Maîtriser les erreurs	15
6.5	Tests de non-régression	15
7	Conclusion	16

1 Introduction et contexte du laboratoire

1.1 Présentation de l'exercice

Le laboratoire *SQL injection UNION attack, determining the number of columns* étudie une vulnérabilité d'injection SQL présente dans le filtre de catégories d'une boutique en ligne. La valeur du paramètre `category` est intégrée à une requête SQL côté serveur sans séparation fiable entre la syntaxe de la requête et la donnée fournie par le client.

L'objectif de l'intervention est d'identifier le nombre exact de colonnes renvoyées par la requête SQL originale. Cette information constitue un prérequis essentiel à une attaque UNION, car toutes les requêtes combinées par l'opérateur UNION doivent présenter un nombre compatible de colonnes.

1.2 Objectif technique

La méthodologie consiste à injecter successivement une, deux puis trois expressions NULL :

```
' UNION SELECT NULL--  
' UNION SELECT NULL,NULL--  
' UNION SELECT NULL,NULL,NULL--
```

La première charge qui ne provoque plus d'erreur révèle le nombre de colonnes attendu par la requête initiale. Dans le laboratoire, la réponse valide est obtenue avec trois valeurs NULL.

Pourquoi utiliser NULL ?

La valeur SQL NULL est généralement compatible avec plusieurs types de données. Elle constitue donc un excellent marqueur neutre pour tester le nombre de colonnes sans devoir connaître immédiatement leur type exact. Une fois le nombre établi, des marqueurs textuels peuvent être utilisés pour rechercher les colonnes capables d'afficher des chaînes.

1.3 Périmètre et résultat attendu

Élément	Valeur
Cible	laboratoire PortSwigger Web Security Academy
Point d'entrée	paramètre <code>category</code> de <code>/filter</code>
Technique	injection SQL UNION SELECT
Outil principal	Burp Suite Proxy et Repeater
Signal d'échec	réponse 500 <code>Internal Server Error</code>
Signal de succès	réponse 200 OK et affichage normal
Résultat	requête originale composée de trois colonnes

Table 1 – Synthèse du scénario d'intervention.

Périmètre autorisé

Les charges et analyses présentées sont limitées au laboratoire conçu à cet effet. Une injection SQL ne doit jamais être testée sur un système tiers sans autorisation explicite.

2 Phase de reconnaissance et interception

2.1 Étape 1 – Accès initial à la plateforme (09 :44)

L'environnement de laboratoire est ouvert dans le navigateur. La page d'accueil présente une boutique fictive et plusieurs catégories de produits. La sélection d'une catégorie modifie dynamiquement le catalogue, ce qui indique qu'une valeur contrôlée par l'utilisateur est transmise au serveur afin de filtrer les résultats.

Cette première observation permet d'identifier le filtre comme point d'entrée potentiel. Le test reste cependant purement fonctionnel à ce stade : aucune conclusion de sécurité ne peut être établie avant l'analyse de la requête HTTP réellement envoyée.

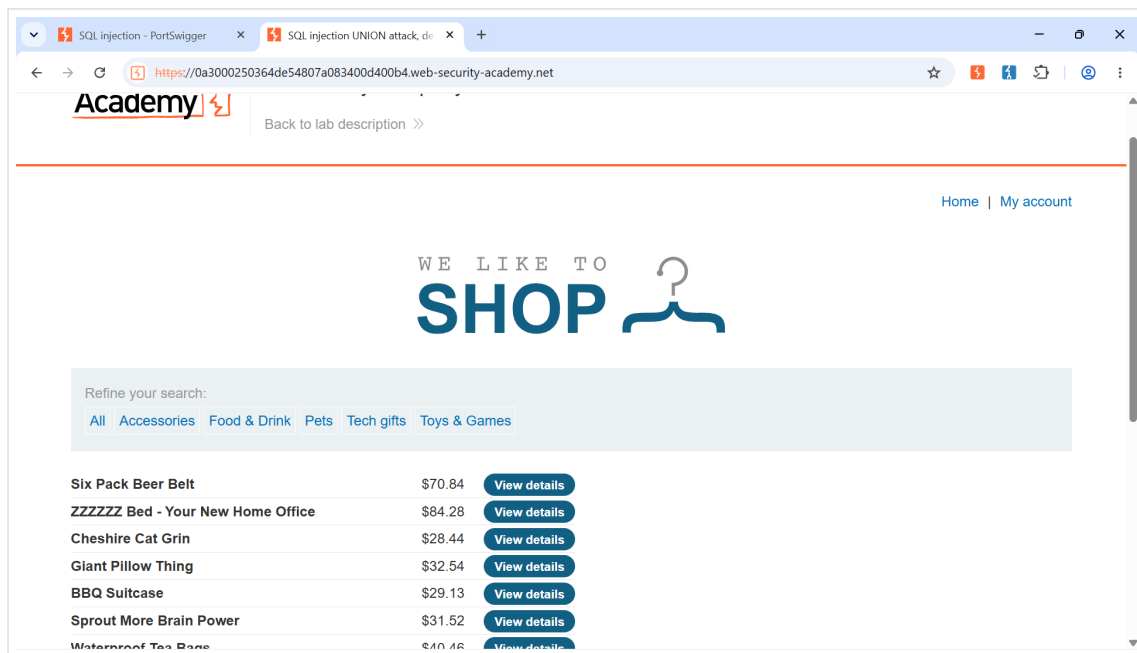


Figure 1 – Accès initial au laboratoire et découverte de l'interface e-commerce à 09 :44.

Référence de preuve : Labo_1_injection_sql/Capture_1_accesaulabo.png.

2.2 Étape 2 – Analyse du trafic HTTP (09 :46)

L'historique HTTP de Burp Proxy est consulté après la sélection d'une catégorie. Une requête GET visant l'endpoint `/filter` est isolée. La chaîne de requête contient le paramètre `category`, dont la valeur correspond à la catégorie choisie dans l'interface.

La présence de cette donnée dans l'URL en fait une candidate naturelle pour un test d'injection. L'historique permet également de conserver la requête légitime de référence, indispensable pour comparer les réponses après chaque modification.

```
GET /filter?category=<CATEGORIE> HTTP/2
Host: <LAB>.web-security-academy.net
```

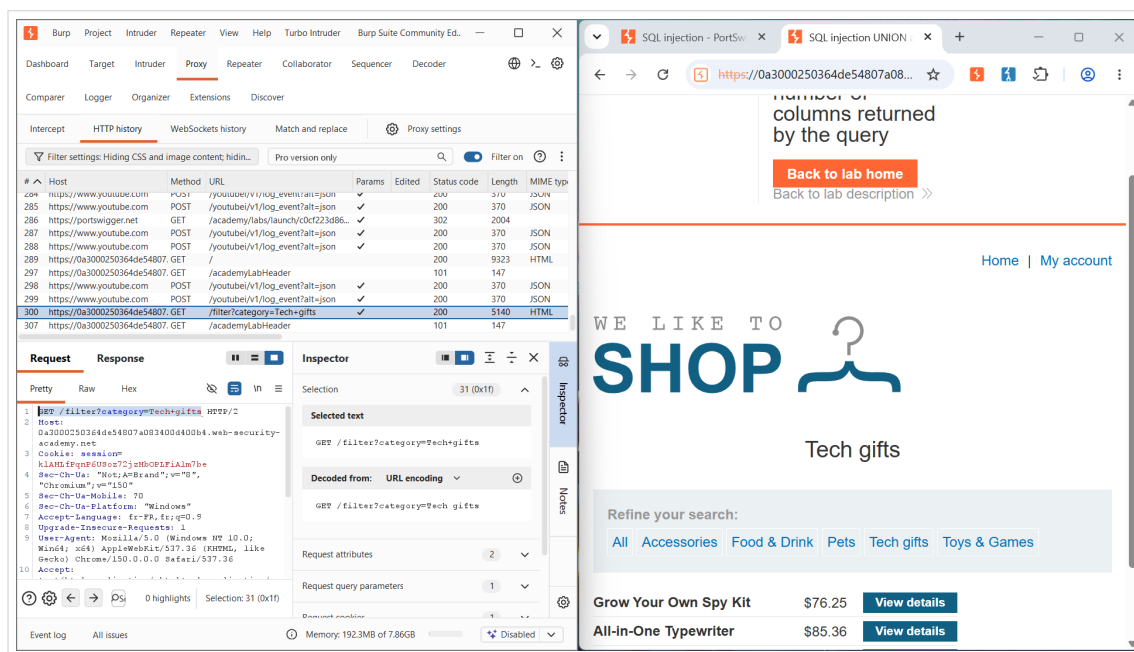


Figure 2 – Repérage de la requête `GET /filter?category=...` dans l'historique HTTP à 09 :46.

Référence de preuve : Labo_1_injection_sql/Capture_2_repÃlerlarequÃteget.png.

2.3 Étape 3 – Transfert vers Burp Repeater (09 :47)

La requête de filtrage est envoyée vers Burp Repeater. Cet outil permet de rejouer manuellement le même échange, de modifier uniquement le paramètre ciblé et de comparer rapidement les codes HTTP, les en-têtes et le corps de la réponse.

Ce transfert évite de répéter toute la navigation dans le navigateur. Il garantit également une démarche contrôlée : un seul élément est modifié entre deux essais, ce qui facilite l'interprétation des résultats.

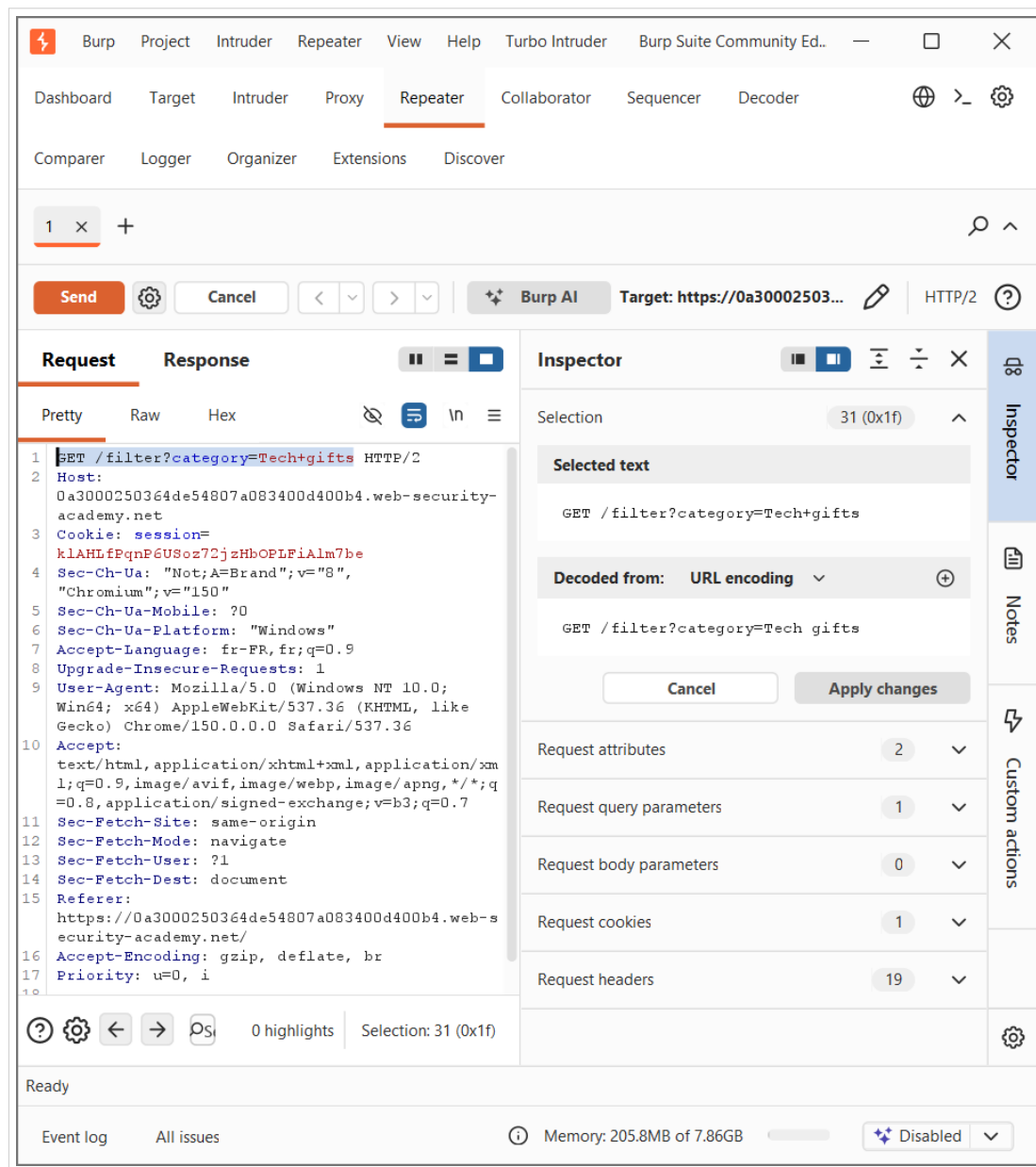


Figure 3 – Envoi de la requête de filtrage vers Burp Repeater à 09 :47.

Référence de preuve : Labo_1_injection_sql/Capture_3_envoi_repeteur.png.

Comportement sécurisé attendu

La valeur de `category` devrait être transmise comme paramètre typé d'une requête préparée. Elle ne devrait jamais pouvoir modifier le nombre de colonnes, ajouter un opérateur UNION ou commenter la fin de l'instruction SQL.

3 Phase d'exploitation et tâtonnement itératif

3.1 Principe de l'approche incrémentale

L'opérateur SQL UNION combine les résultats de deux requêtes SELECT. Pour que l'opération soit valide, les deux projections doivent renvoyer le même nombre de colonnes et présenter des types compatibles à chaque position.

La méthode de découverte suit donc une progression simple : commencer avec une colonne NULL, puis ajouter une nouvelle colonne à chaque tentative jusqu'à la disparition de l'erreur. Les échecs ne sont pas des incidents inattendus ; ils constituent des mesures nécessaires pour déterminer la structure de la requête originale.

3.2 Étapes 4 et 5 – Test avec une colonne (09 :51–09 :52)

La première charge injectée est la suivante :

```
' +UNION+SELECT+NULL--
```

Après décodage URL, la valeur correspond conceptuellement à :

```
' UNION SELECT NULL--
```

L'apostrophe ferme la chaîne initiale, UNION SELECT NULL ajoute une seconde requête composée d'une seule colonne, puis le commentaire – neutralise la suite de la requête originale. La capture montre la modification du paramètre dans Repeater.

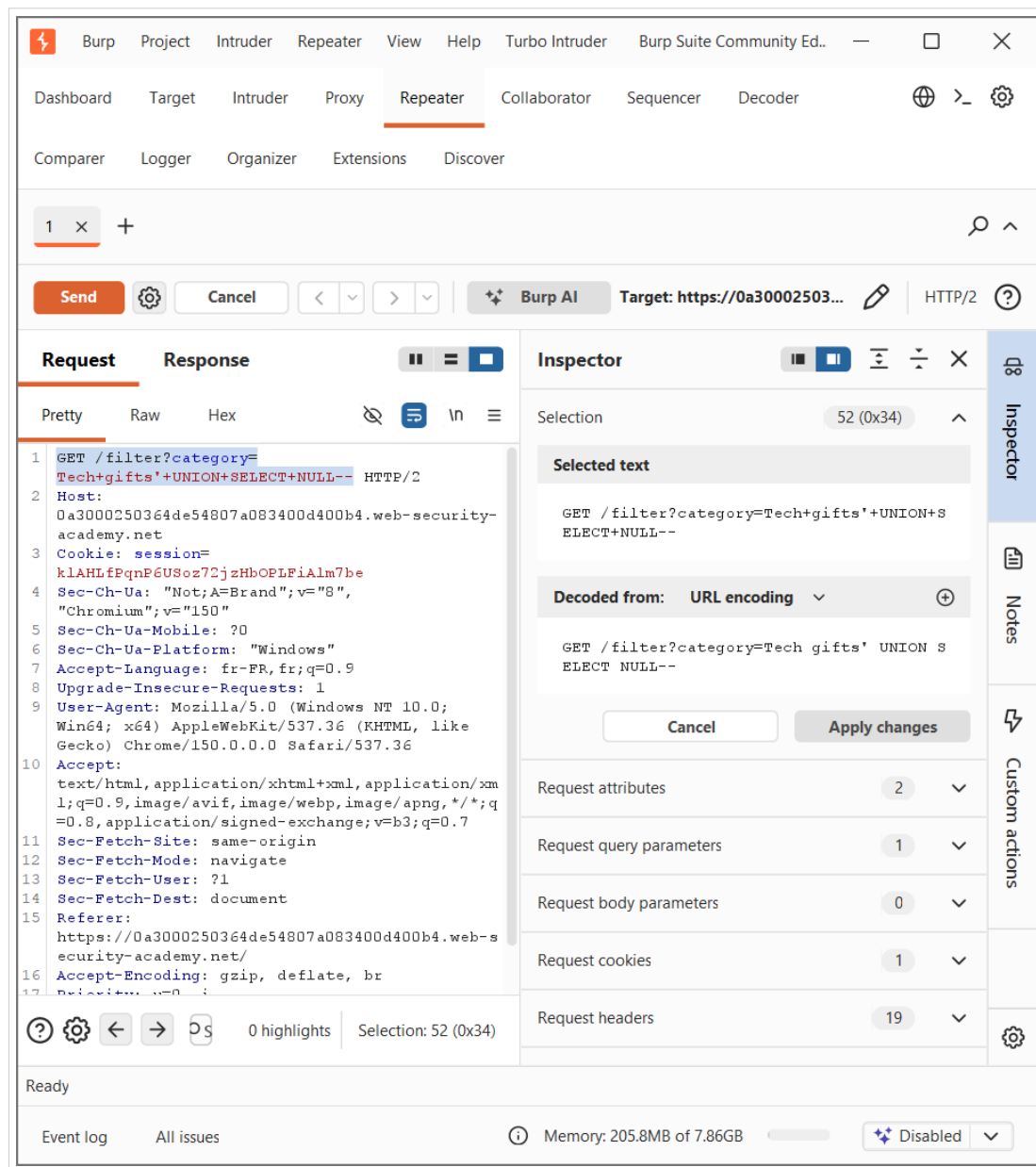


Figure 4 – Modification de la requête avec une seule colonne NULL à 09 :51.

Référence de preuve : Labo_1_injection_sql/Capture_4_modification_du_requete.png.

Le serveur répond par 500 Internal Server Error. Dans ce contexte contrôlé, ce résultat indique que la requête ajoutée ne correspond pas à la structure attendue. La cause la plus probable est un nombre de colonnes insuffisant : la requête originale renvoie plus d'une colonne.

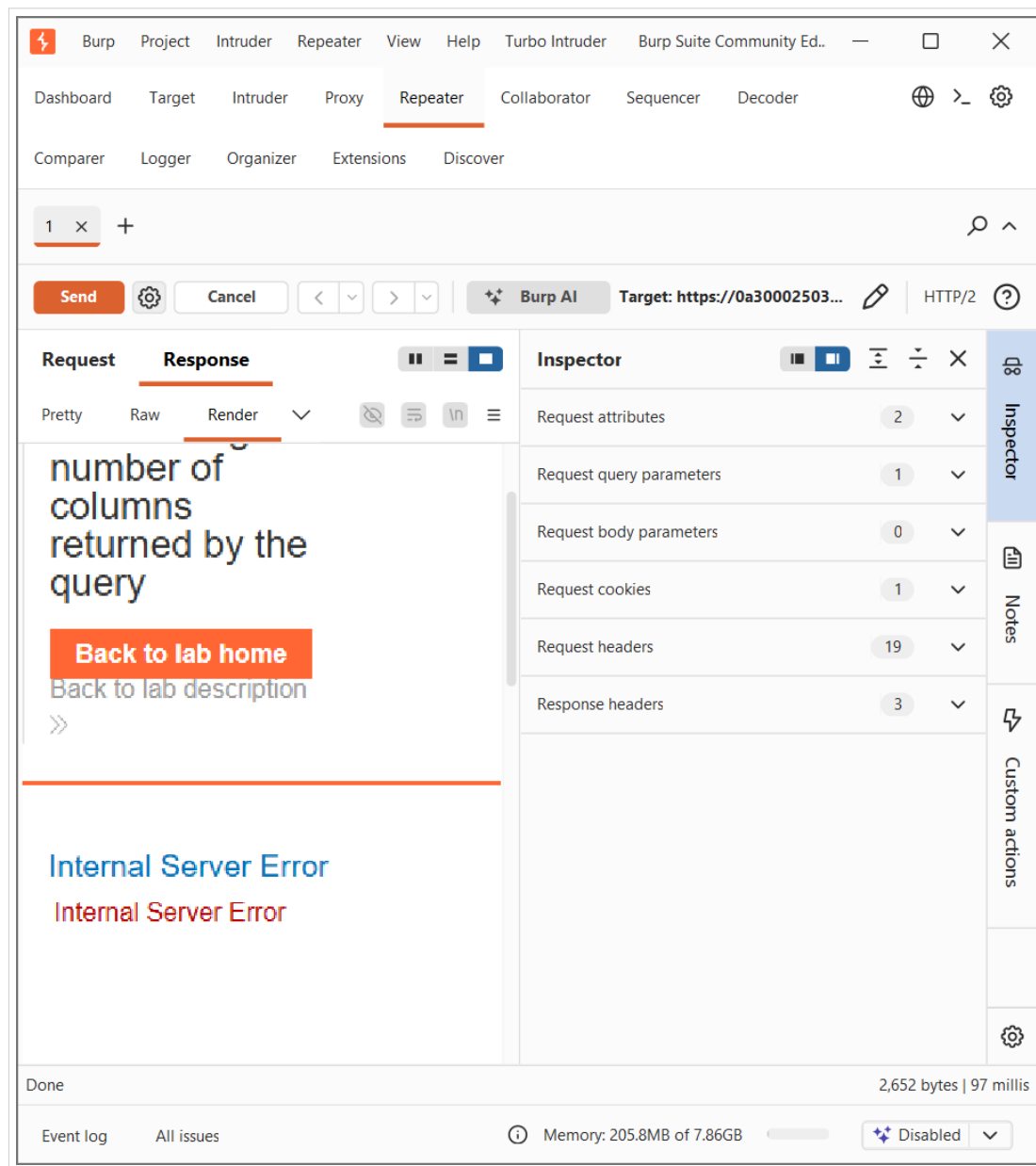


Figure 5 – Réponse 500 *Internal Server Error* après le test avec une colonne à 09 :52.

Référence de preuve : Labo_1_injection_sql/Capture_5_resultat_injection_erreur_500.png.

Interprétation du code 500

Un code 500 ne prouve pas à lui seul une injection SQL sur une application inconnue. Il peut résulter de nombreuses exceptions. Ici, sa valeur vient de la comparaison itérative : la même requête redevient valide lorsque le bon nombre de colonnes est utilisé. Cette corrélation confirme le désalignement structurel des premiers essais.

3.3 Étapes 6 et 7 – Test avec deux colonnes (09 :54–09 :55)

Une seconde valeur NULL est ajoutée afin de tester une projection à deux colonnes :

```
' + UNION + SELECT + NULL, NULL --
```

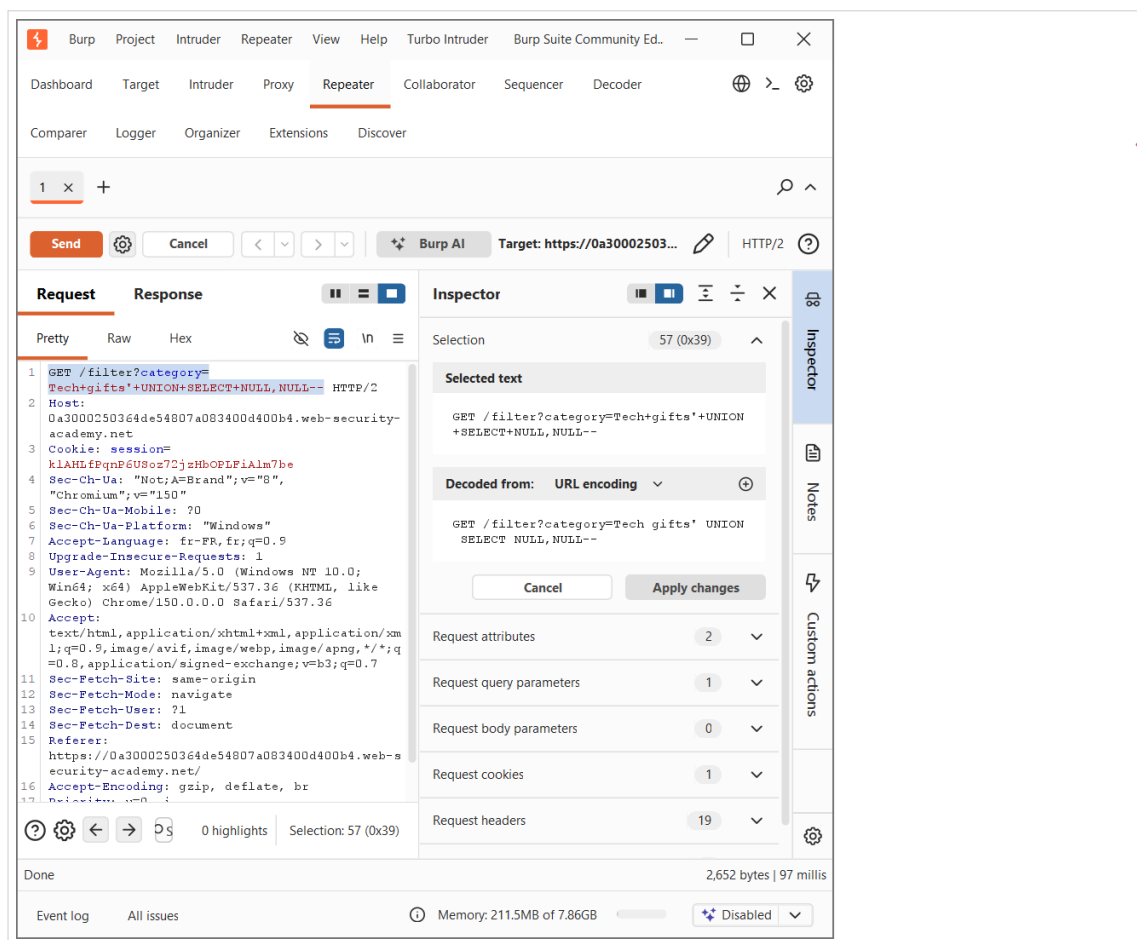


Figure 6 – Nouvel essai avec deux valeurs NULL à 09 :54.

Référence de preuve : Labo_1_injection_sql/Capture_6_injection_testeravec2colonnes.png.

La réponse reste un 500 Internal Server Error. La persistance du même signal signifie que deux colonnes ne suffisent toujours pas à correspondre à la projection originale. L'hypothèse est donc affinée : la requête cible renvoie au minimum trois colonnes.

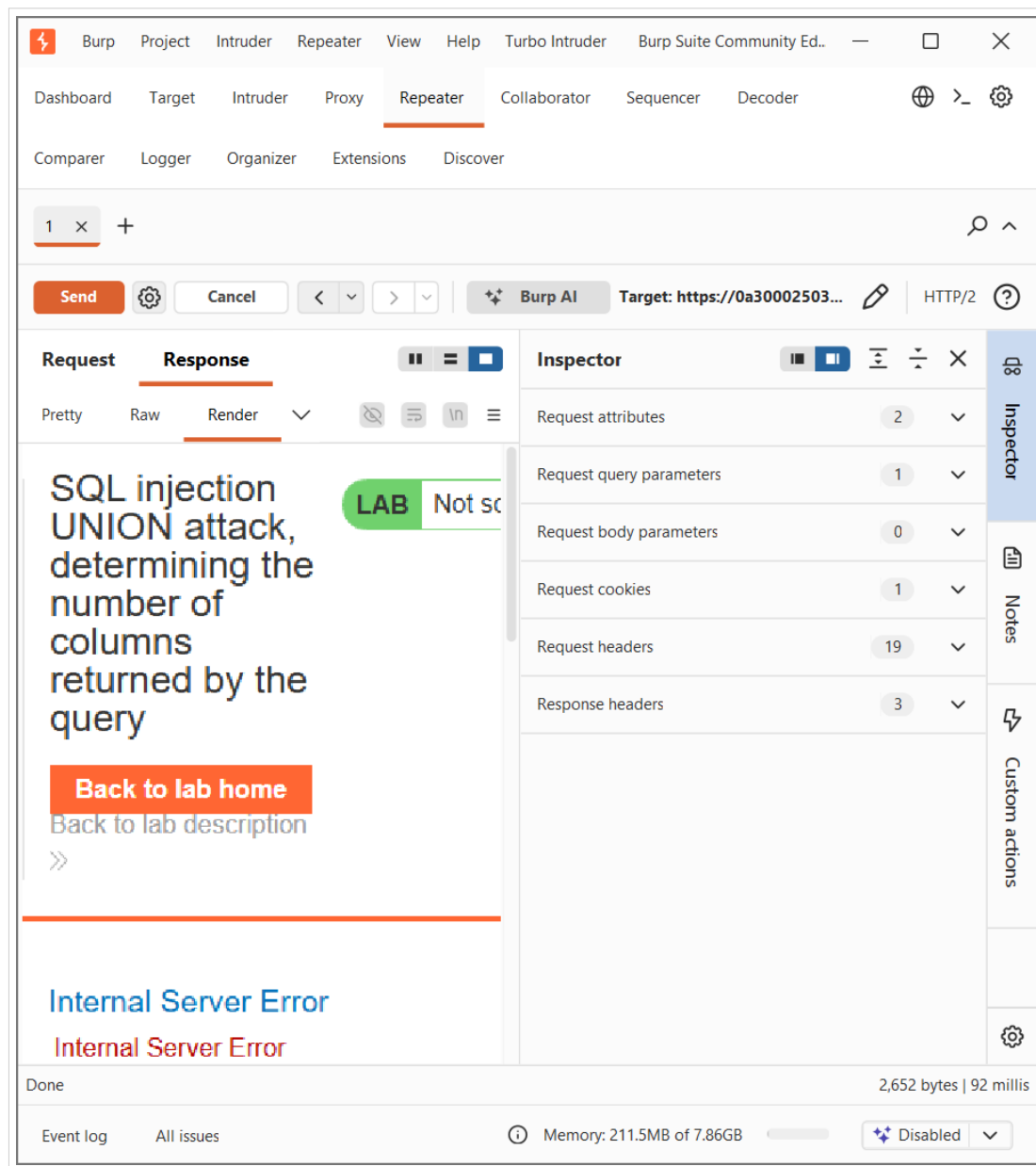


Figure 7 – Persistance de l'erreur 500 après le test avec deux colonnes à 09 :55.

Référence de preuve : Labo_1_injection_sql/Capture_7_injection_testeravec2colonnes_erreur500.png.

3.4 Étapes 8 et 9 – Validation avec trois colonnes (09 :57)

La charge est étendue à trois valeurs NULL :

```
' + UNION + SELECT + NULL, NULL, NULL --
```

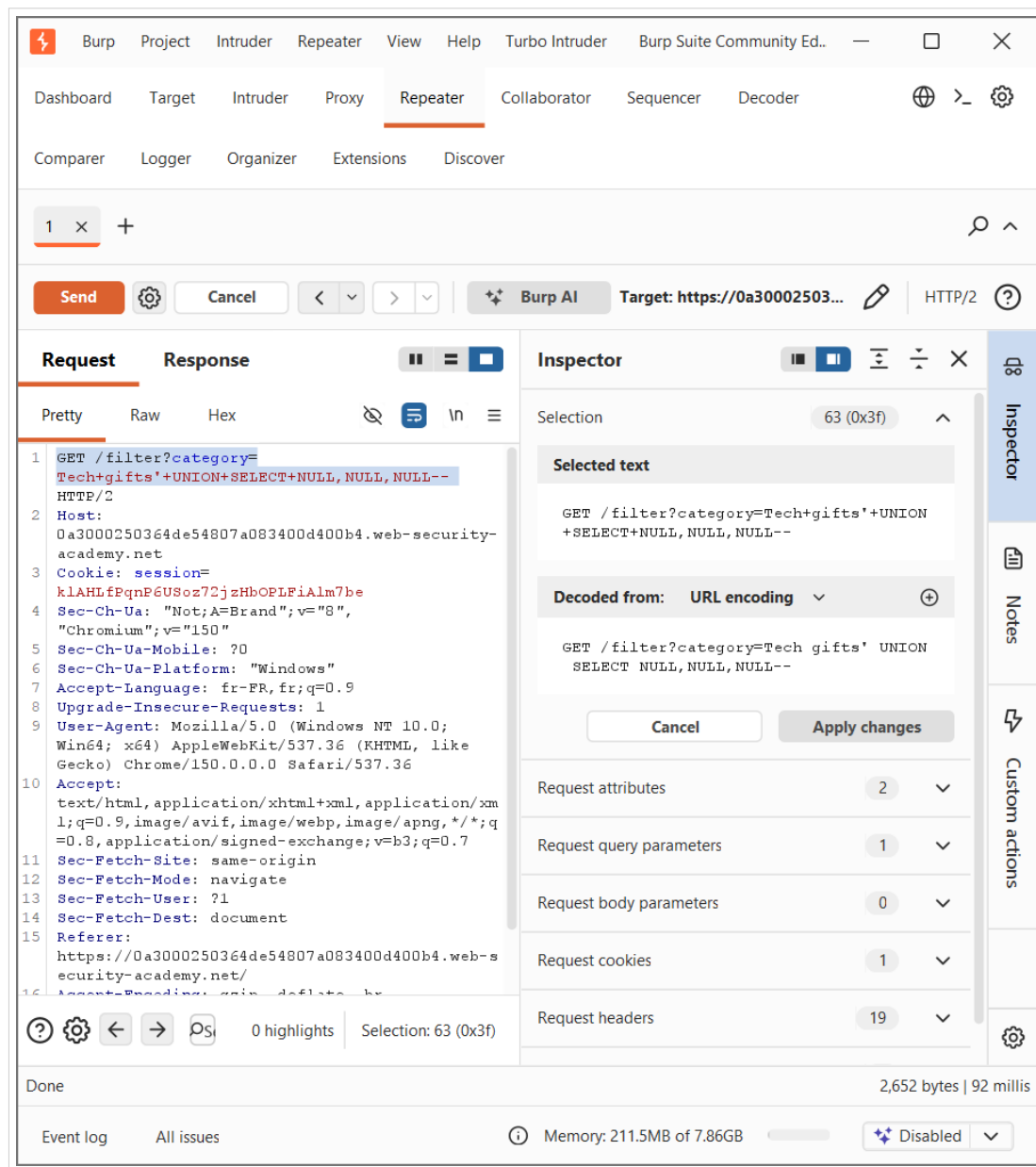


Figure 8 – Injection d'une projection composée de trois valeurs NULL à 09 :57.

Référence de preuve : Labo_1_injection_sql/Capture_8_injection_testeravec3colonnes.png.

Cette fois, le serveur répond par 200 OK. Le corps contient la page normale du catalogue et les lignes produites par la requête UNION. La disparition de l'erreur démontre que la projection ajoutée correspond désormais au nombre de colonnes de la requête initiale.

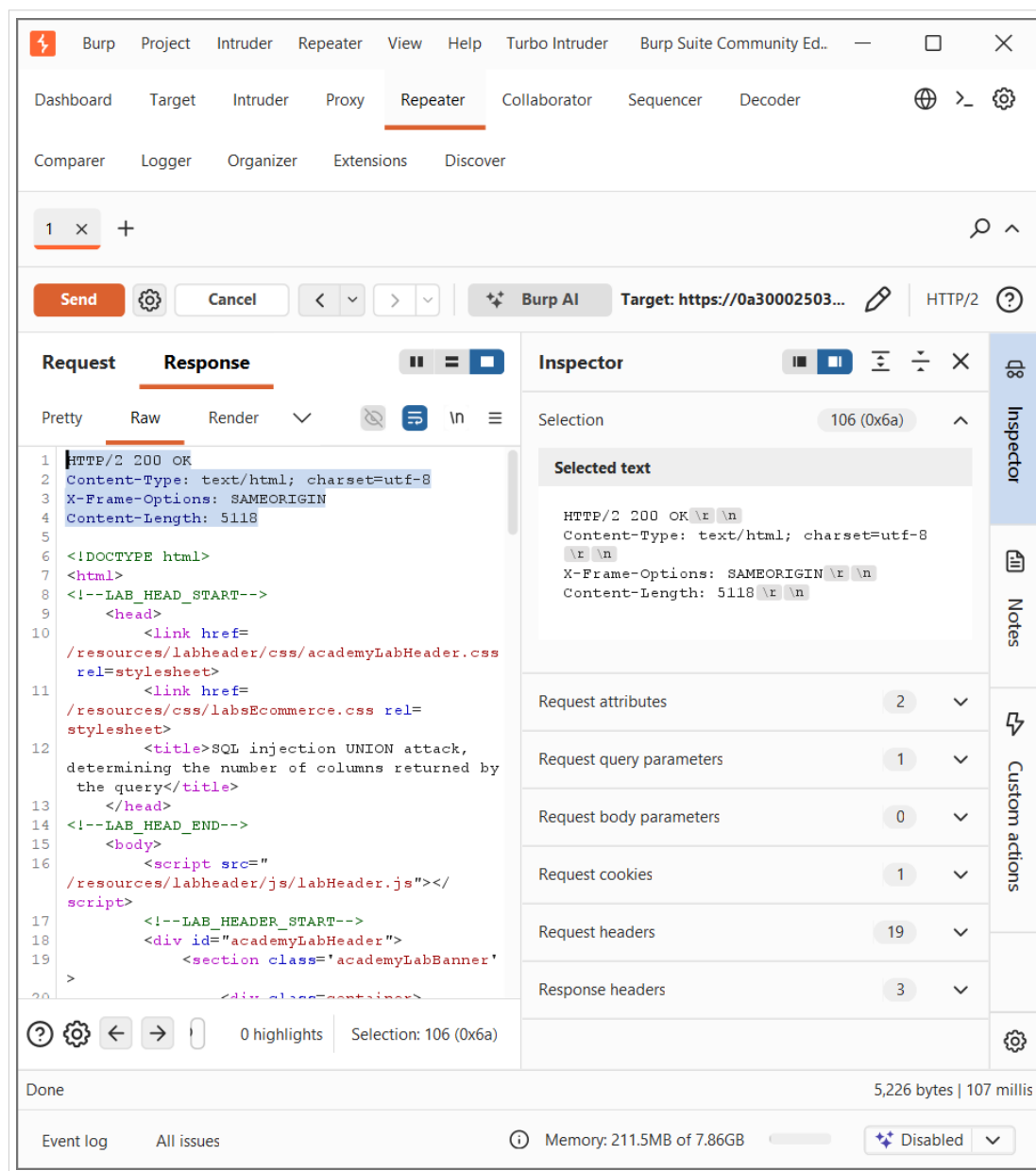


Figure 9 – Réponse 200 OK confirmant une projection à trois colonnes à 09 :57.

Référence de preuve :

Labo_1_injection_sql/Capture_9_injection_testeravec3colonnesreussis.png.

Signification du code 200

Le code 200 OK est déterminant lorsqu'il est comparé aux deux réponses 500. Il indique que la requête SQL résultante a été acceptée et que la page a pu être générée normalement. L'affichage des lignes supplémentaires renforce cette preuve. Le nombre exact de colonnes est donc trois.

3.5 Synthèse comparative des essais

Essai	Charge utile	Réponse	Interprétation
1	'+UNION+SELECT+NULL-	500	une colonne insuffisante
2	'+UNION+SELECT+NULL,NULL-	500	deux colonnes insuffisantes
3	'+UNION+SELECT+NULL,NULL,NULL-	200	trois colonnes : structure valide

Table 2 – Comparaison des trois tentatives de détermination du nombre de colonnes.

Résultat technique confirmé

La requête SQL utilisée par le filtre renvoie exactement **trois colonnes**.

4 Clôture et validation du laboratoire

4.1 Étape 10 – Validation officielle (09 :58)

Après la réponse réussie avec trois colonnes, la plateforme Web Security Academy marque le laboratoire comme résolu. Le bandeau de félicitations confirme que la méthodologie employée satisfait l'objectif pédagogique du scénario.

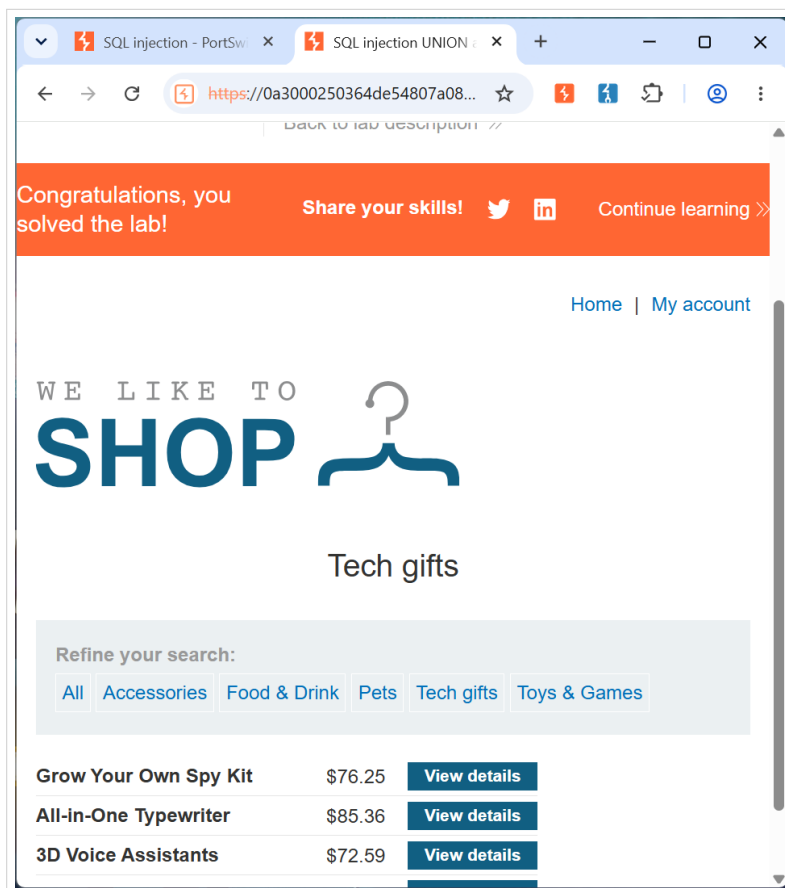


Figure 10 – Validation officielle du laboratoire et affichage du statut résolu à 09 :58.

Référence de preuve : Labo_1_injection_sql/Capture_10_laboreussis.png.

4.2 Chronologie de l'intervention

Heure	Phase	Action
09 :44	Reconnaissance	ouverture du laboratoire
09 :46	Interception	identification de <code>/filter?category=</code>
09 :47	Préparation	transfert vers Repeater
09 :51–09 :52	Exploitation	essai à une colonne, réponse 500
09 :54–09 :55	Exploitation	essai à deux colonnes, réponse 500
09 :57	Validation	essai à trois colonnes, réponse 200
09 :58	Clôture	laboratoire officiellement résolu

Table 3 – Chronologie synthétique des opérations réalisées.

5 Analyse du risque

5.1 Vulnérabilité confirmée

Le paramètre `category` permet de modifier la structure de la requête SQL et d'ajouter une seconde projection avec `UNION SELECT`. La découverte du nombre de colonnes constitue la première étape d'une exploitation plus large : un attaquant pourrait ensuite rechercher les colonnes textuelles, cartographier les tables accessibles et tenter de récupérer des données sensibles.

5.2 Impacts potentiels

- **Confidentialité** : récupération d'utilisateurs, de secrets, de données métier ou de métadonnées du schéma ;
- **Intégrité** : selon le contexte et les privilèges, altération de données au moyen d'autres formes d'injection ;
- **Disponibilité** : requêtes coûteuses, erreurs répétées ou opérations destructrices si le compte possède des droits excessifs ;
- **Contrôle d'accès** : exposition de données normalement filtrées ou appartenant à d'autres utilisateurs.

5.3 Sévérité

Le risque est qualifié d'**élevé**. Le laboratoire démontre une injection SQL exploitable à distance et une capacité à contrôler une seconde requête `SELECT`. L'impact final dépendrait des privilèges accordés au compte de base et des données accessibles.

Critère	Évaluation
Vecteur	paramètre HTTP accessible à distance
Authentification	non requise dans le scénario
Complexité	faible à modérée
Interaction utilisateur	aucune
Exploit démontré	ajout d'une requête UNION à trois colonnes
Extension possible	découverte de types, tables, colonnes et données

Table 4 – Qualification technique de la vulnérabilité.

6 Recommandations techniques et remédiation

6.1 Utiliser des requêtes préparées

La correction principale consiste à définir la structure SQL à l'avance et à transmettre la catégorie comme une valeur liée. La donnée ne peut alors plus fermer la chaîne, ajouter UNION ou introduire un commentaire.

```
SELECT id, name, price
FROM products
WHERE category = ? AND released = ?
```

Exemple conceptuel en Java :

```
String sql = "SELECT id, name, price "
            + "FROM products "
            + "WHERE category = ? AND released = ?";

PreparedStatement ps = connection.prepareStatement(sql);
ps.setString(1, category);
ps.setBoolean(2, true);
ResultSet results = ps.executeQuery();
```

6.2 Valider les catégories autorisées

Si le catalogue possède une liste fermée de catégories, l'application doit vérifier que la valeur reçue appartient à cette liste. Cette validation améliore la cohérence métier et réduit les entrées inattendues, mais elle ne remplace pas la requête préparée.

6.3 Appliquer le moindre privilège

Le compte SQL de l'application doit uniquement pouvoir lire les tables nécessaires au catalogue. Il ne doit pas posséder de droits d'administration, de création d'objets ni d'accès aux tables d'authentification si la fonction ne les utilise pas.

6.4 Maîtriser les erreurs

Les erreurs SQL détaillées doivent être conservées dans des journaux protégés et remplacées par une réponse générique côté client. Cette mesure limite la divulgation, mais elle ne corrige pas la concaténation vulnérable.

6.5 Tests de non-régression

Après correction, les scénarios suivants doivent être automatisés :

1. vérifier que les catégories légitimes retournent toujours le catalogue attendu ;
2. envoyer une apostrophe et confirmer qu'elle est traitée comme une donnée ou rejetée proprement ;
3. rejouer les trois charges UNION SELECT NULL et vérifier qu'elles ne modifient jamais la requête ;

4. contrôler l'absence de différence exploitable entre conditions SQL vraies et fausses ;
5. vérifier que les erreurs ne révèlent ni requête, ni table, ni moteur de base ;
6. confirmer que le compte SQL possède uniquement les permissions requises.

Comportement sécurisé attendu

Après correction, la chaîne `'+UNION+SELECT+NULL,NULL,NULL-` doit être considérée comme une valeur littérale invalide ou comme un nom de catégorie inexistant. Elle ne doit jamais produire une seconde requête `SELECT`.

7 Conclusion

L'intervention a permis de déterminer méthodiquement le nombre de colonnes de la requête vulnérable. L'essai avec une colonne a produit une réponse 500, tout comme l'essai avec deux colonnes. L'ajout d'une troisième valeur NULL a rétabli une réponse 200 OK et un rendu normal de la page.

Cette comparaison confirme que la requête SQL initiale renvoie exactement trois colonnes. Les erreurs 500 ont constitué des signaux de tâtonnement attendus, tandis que le passage au code 200 a représenté le moment décisif de la validation.

La vulnérabilité est due à la concaténation d'une entrée utilisateur dans la requête SQL. Sa correction repose en priorité sur les requêtes préparées, complétées par une validation métier, le moindre privilège, une gestion sûre des erreurs et des tests automatisés.

Bilan final

Le laboratoire est résolu. La séquence 1 colonne = 500, 2 colonnes = 500, puis 3 colonnes = 200 démontre que l'injection UNION doit comporter trois expressions pour être compatible avec la requête originale.